

MÓDULO 1 · CURSO IAC UCM

Introducción a Terraform

1 - ¿Qué es Terraform?

1. ¿Qué es Terraform?

2. Componentes de Terraform

3. Elementos básicos de HCL

4. Cómo funciona por dentro

5. Flujo de trabajo Core

6. Estado y backend remoto

Tres ideas para empezar

Terraform (HashiCorp, 2014) es **laC**: escribes el **estado final deseado**. Terraform calcula las operaciones.

Declarativo

Dices qué quieres (una VPC, una subred). El cómo lo resuelve la herramienta.

Providers

Plugins que hablan con una API: AWS, Azure, GCP, Kubernetes. Sin provider, no hay gestión.

Estado

Fichero que recuerda qué objetos reales corresponden a lo declarado en el código.

HCL y un mismo flujo

El lenguaje es HCL

- HashiCorp Configuration Language
- Más legible que JSON y más acotado que un lenguaje general
- Los ficheros de configuración son `.tf`

Para qué se creó

- Unificar proveedores en un flujo versionable y repetible
- El ciclo es el mismo en cualquier nube
- Los tipos (`aws_instance` , `google_compute_instance`) siguen siendo de cada proveedor

La infra se trata como software

Versiones

El diseño vive en Git: historial, revisión de cada cambio y el mismo despliegue otra vez.

Consistencia

Desarrollo, pruebas y producción salen del mismo código. Cambian los valores, no el procedimiento.

Agilidad

Un cambio se previsualiza y se aplica igual cada vez, sin reescribir los pasos en la consola.

2 - Componentes de Terraform

1. ¿Qué es Terraform?
- 2. Componentes de Terraform**
3. Elementos básicos de HCL
4. Cómo funciona por dentro
5. Flujo de trabajo Core
6. Estado y backend remoto

Piezas que usan la configuración

Los `.tf` describen el estado deseado. Tres piezas trabajan con ellos.

CLI

Terraform Core

Binario de la CLI. Construye el grafo, calcula el plan y ejecuta el apply.

AWS

Provider

Plugin en tu máquina. Traduce el HCL a una petición concreta del proveedor.

Σ

Estado

`terraform.tfstate`. Mapea el código con los objetos que ya existen.

Provider local y API remota

Provider · en tu máquina

- `terraform init` lo descarga (hashicorp/aws ...)
- Conoce los tipos `resource` y `data`
- Recibe el trabajo de Core y llama a la API

API · en internet

- Servicio de la nube: AWS, Azure, GCP
- El plugin abre la conexión HTTPS
- Devuelve ids, direcciones o errores

Qué hay en los archivos .tf

HCL versionado en Git. El nombre solo organiza (`main` , `variables` , `outputs` , `providers` , `versions`): Terraform lee **todos** los `.tf` del directorio.

Bloque	Qué representa
<code>terraform</code>	Versión mínima y providers requeridos
<code>provider</code>	Cómo conectar: región, cuenta
<code>variable</code> / <code>locals</code>	Entradas y valores internos
<code>data</code>	Consulta lo que ya existe, sin crearlo
<code>resource</code>	Objeto que Terraform crea y gestiona
<code>output</code>	Valores expuestos tras el <code>apply</code>

Resource

Qué gestiona

Objeto que Terraform crea, actualiza y puede destruir: VPC, instancia, bucket.

Sintaxis

```
resource "TIPO" "NOMBRE_LOCAL"
```

Dos nombres

El tipo viene del provider (`aws_vpc`). El nombre local solo existe en tu código, no en AWS.

El bloque data solo lee

La variable puede ir en el filtro: es el criterio. El `data` es la consulta, por ejemplo `data.aws_ami.ubuntu.id`.

Solo lectura

No crea, no modifica y no destruye. El objeto no pertenece a este proyecto.

En cada plan

El provider pregunta a la API y rellena un id, un nombre o una IP.

Caso típico: AMI

En Git va el criterio (Canonical, Ubuntu 22.04, la más reciente). El `ami-...` lo devuelve AWS.

Quién produce cada valor

Bloque	Quién lo produce	Con la nube
variable	Tú: <code>default</code> , <code>.tfvars</code> o <code>-var</code>	Nada
local	El código, con valores que ya tiene	Nada
data	La API, durante el <code>plan</code>	Solo lee
output	Un resource o un data, tras el <code>apply</code>	Publica el resultado
resource	Tú declaras; la API lo materializa	Crea, actualiza y puede destruir

Estado: terraform.tfstate

Mapeo

JSON entre lo declarado y los objetos reales: IDs, ARNs, IPs.

Sin él no hay diff

Terraform no sabría qué existe ya ni qué hay que cambiar.

Sensible

No se edita a mano ni se sube a Git: puede contener secretos. En equipo, backend remoto.

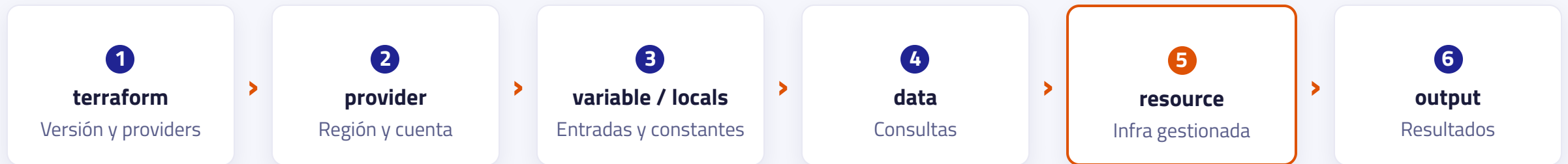
Mapa mental

Componente	Pregunta que responde
Configuración <code>.tf</code>	¿Qué infraestructura quiero?
Provider	¿Qué plugin local traduce el HCL a peticiones?
Resource	¿Qué objeto concreto creo y gestiono?
Data source	¿Qué bloque <code>data</code> lee un objeto que ya existe?
Estado	¿Qué existe ya y cómo se relaciona con mi código?
Core	¿Cómo calculo y aplico los cambios?

3 - Elementos básicos de HCL

1. ¿Qué es Terraform?
2. Componentes de Terraform
- 3. Elementos básicos de HCL**
4. Cómo funciona por dentro
5. Flujo de trabajo Core
6. Estado y backend remoto

Orden de lectura de los bloques



Bloque terraform

```
terraform {  
  required_version = ">= 1.5.0"  
  required_providers {  
    aws = {  
      source  = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}
```

Operadores de versión

Operador	Ejemplo	Significado
=	= 5.0.0	Solo esa versión exacta
>=	>= 5.0	Esa versión o superior
~>	~> 5.0	Admite 5.1, 5.2... y bloquea el salto a 6.0

Provider

Región del provider

```
provider "aws" {  
  region = "eu-west-1"  
}
```

Región de los resources

Si fijas la región aquí y un resource no indica otra, Terraform usa `eu-west-1`.

Locals: constantes internas

```
locals {  
  etiquetas_comunes = {  
    Proyecto    = "UCM-DevOps-IaC"  
    Asignatura  = "Arquitectura en la Nube"  
    Gestionado  = "Terraform"  
  }  
}
```

variable

Entrada. La aportas al ejecutar (`default` , `.tfvars` o `-var`) y puede cambiar por entorno.

local

Valor interno. Lo compone el código con datos que ya tiene. No se pasa desde fuera.

Variable o local

El tipo de instancia cambia por entorno. Las etiquetas repetidas, no.

variable

- Entrada: la fijas al ejecutar
- Llega por `default`, `.tfvars` o `-var`
- Se usa como `var.tipo_instancia`

local

- Valor interno del código
- Lo compone con datos que ya tiene. No se pasa desde fuera
- Se usa como `local.etiquetas_comunes`

Data: el criterio va en el código

```
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"] # Canonical

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}
```

De dónde sale cada atributo

`data.aws_ami.ubuntu.id`

- Lo devuelve la API en cada `plan`
- En Git está el criterio, no el `ami-...`

`var.tipo_instancia`

- Lo eliges tú
- Puede cambiar por entorno (`.tfvars` , `-var`)

Resource: lo que Terraform gestiona

```
resource "aws_instance" "servidor_web" {  
  ami          = data.aws_ami.ubuntu.id  
  instance_type = var.tipo_instancia  
  
  tags = merge(local.etiquetas_comunes, {  
    Name = "EC2-Servidor-Web-AWS"  
  })  
}
```

Output: lo que queda visible

```
output "ec2_public_ip" {  
    description = "Dirección IP pública del servidor EC2"  
    value       = aws_instance.servidor_web.public_ip  
}
```

4 - Cómo funciona por dentro

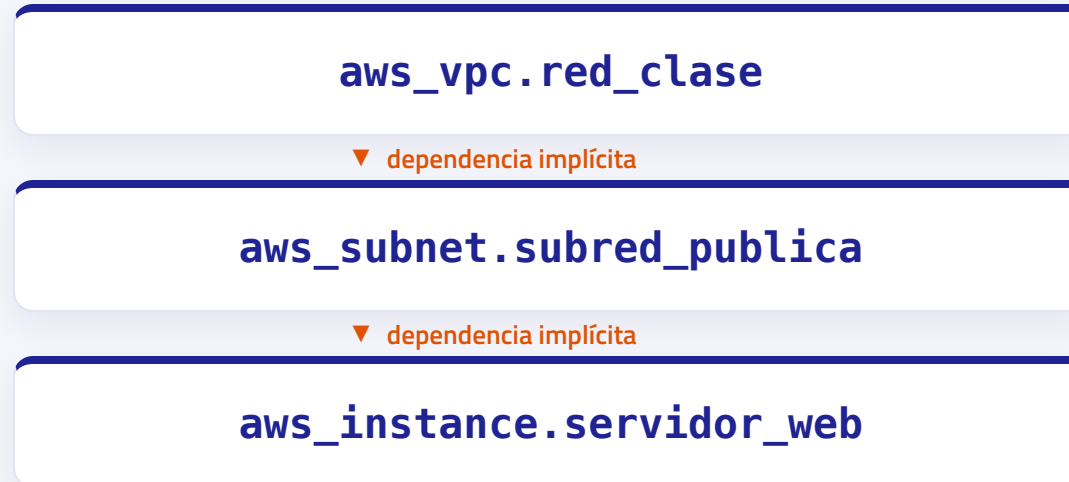
1. ¿Qué es Terraform?
2. Componentes de Terraform
3. Elementos básicos de HCL
- 4. Cómo funciona por dentro**
5. Flujo de trabajo Core
6. Estado y backend remoto

Del HCL a la API



Core también lee y escribe `terraform.tfstate`. La conexión con la nube la abre el plugin.

Grafo de dependencias



- **Implícita:** una referencia, `vpc_id = aws_vpc.red_clase.id`
- **Explícita:** `depends_on`, si el código no muestra el vínculo
- Sin dependencia mutua, hasta **10** operaciones en paralelo

Configuración, estado y realidad

Configuración

Estado deseado en los `.tf`

vs

Estado registrado

`terraform.tfstate`

vs

Infra real

Lo que responden las APIs

Antes del plan, Terraform refresca el estado contra la API. Un cambio a mano en la consola es **deriva** (configuration drift).

Apply actualiza el estado

1 • Llamadas a la API

El provider traduce el plan. Ejemplo en EC2: `RunInstances` .

2 • Estado al día

Si la API responde bien, Core guarda IDs, IPs y ARNs.

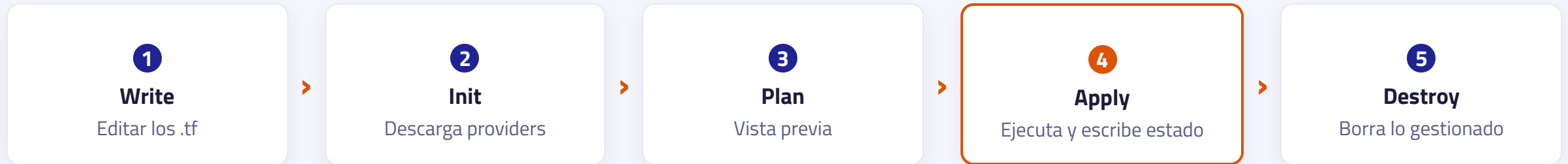
AGENDA

5 - Flujo de trabajo Core

1. ¿Qué es Terraform?
2. Componentes de Terraform
3. Elementos básicos de HCL
4. Cómo funciona por dentro
- 5. Flujo de trabajo Core**
6. Estado y backend remoto

Write, plan, apply

HashiCorp resume **Write** → **Plan** → **Apply**. En clase añadimos `init` al arrancar y `destroy` al limpiar.



Qué hace cada comando

Comando	Qué hace
<code>terraform init</code>	Providers y backend. Obligatorio la primera vez y si cambian.
<code>terraform plan</code>	Compara configuración, estado y realidad. No modifica la nube.
<code>terraform apply</code>	Aplica el plan, pide confirmación y actualiza el estado.
<code>terraform destroy</code>	Elimina los recursos de este proyecto.
<code>fmt</code> / <code>validate</code>	Formato y sintaxis, antes del plan.

AGENDA

6 - Estado y backend remoto

1. ¿Qué es Terraform?
2. Componentes de Terraform
3. Elementos básicos de HCL
4. Cómo funciona por dentro
5. Flujo de trabajo Core
- 6. Estado y backend remoto**

Backend remoto en AWS

En local el estado es `terraform.tfstate`. En equipo o en CI/CD se comparte un backend.

A/B

Varios actores

Desarrolladores y el pipeline escriben el mismo estado.

S3

Bucket S3

Guarda el tfstate, con cifrado y versionado.



DynamoDB

State locking: mientras un `apply` escribe, los demás esperan.

El estado puede llevar secretos en claro. `*.tfstate` y `.terraform/` van en `.gitignore`.

Takeaways

1. Terraform describe el **estado deseado** en HCL.
2. Configuración, provider, resource, estado y Core colaboran, y cada uno tiene un sitio.
3. El flujo es **Write** → **Init** → **Plan** → **Apply**, y **Destroy** limpia.
4. El **estado** es sensible: fuera de Git; en equipo, backend remoto y locking.

Siguiente paso: laboratorio del módulo, con la plantilla.